

A Computer/Human Mastermind Player using Grids

Wayne Goddard

*Dept of Computer Science, University of Natal, Durban 4041, South Africa, and
Dept of Computer Science, Clemson University, Clemson SC 29634, USA
goddard@cs.clemson.edu*

Abstract

This paper introduces a grid-based strategy for determining the secret code in the game Mastermind. The grids allow for a decent human player, while also providing a computer player that always has explicit knowledge of the progress.

Keywords: Mastermind, games, representations

Computing Review Categories: I.2.1, I.2.3, G.2.1

1 Introduction

One of the most enduring problems in AI is the development of computer players for human games. There is still much interest in players for chess, go or bridge. In this paper we consider a much simpler game, namely Mastermind, and investigate the success of a strategy that can be automated but is simple enough for a human to employ.

The game Mastermind (trademark) was introduced with a storm in the early 70s. The simplicity of the game has led to its comeback; now many cell-phones and even some TV sets have a version. In the game, there are two players, CodeBreaker and CodeSetter. The CodeSetter creates a secret code (string) of 4 coloured pegs, each peg one of 6 colours. The CodeBreaker must deduce the secret code by asking a series of questions. Each question is itself a candidate code (“a guess”), and the response is two integers called Black and White. The value of Black indicates in how many positions there is exact agreement, and the value of White indicates how many colours are correct but in the wrong position. Another way to put this is that the sum of Black and White is the maximum value of Black over all permutations of the guess code. (In the actual board game, the response is given by using little pegs.)

There is now an extensive literature on the game. On the mathematical side there are results providing bounds and asymptotics (see e.g. [2, 3, 8]). As far as actual players and computer strategies go, there are two lines of work. One line is for a player using the computer’s calculating abilities. In this approach, the next guess-question is chosen by considering all possibilities and taking the one which “best” splits up the set of remaining possible secrets (see e.g. [7, 14]). The other line is for considering how to implement simple algorithms in Prolog (see e.g. [5, 1, 15]). One exception is the use of genetic algorithms, developed by Merelo et al. [12].

A fundamental difficulty for a human CodeBreaker is to understand all the information that has been revealed so far. Further, humans playing Mastermind can be rather haphazard. The book [13] explores the tortuous logic

needed to try to deduce information from an unstructured series of guesses. The task of gaining knowledge through queries is, of course, an old AI question. In the context of Mastermind, Koyama and Lai [9, 10] were the first to consider it, though their approach of minimising entropy is equivalent to finding the best split.

1.1 Contribution

In this paper we introduce a grid-based representation and strategy which allow for quick assimilation of new information while still providing a decent player. As a human player, it provides a good player, and with practice, can be used very fast. As a computer player, its importance is that the player can explain its progress. Also, the space and time for the single-grid approach is $O(h^2)$ where h is the number of positions, which is considerable smaller than the exponential time and space of most players.

In the following discussion we will use as an example game the original 6-colour 4-position (no blanks) game with $6^4 = 1296$ possible secret codes, though the ideas apply in general. For this version, Knuth [8] used a calculation approach to show that there is a strategy that is guaranteed to take at most 5 steps. Koyama and Lai [11] determined that the optimal expected number of guesses for the original game is 4.340. (For the equivalent values for other versions, see [6].) It is known that a random player (randomly pick any remaining possible code) takes an average of about 4.65.

While Knuth’s strategy can easily be programmed, determining the optimal strategy is computationally challenging; of course, once known, the strategy can be hard-coded with a lookup table. Neither is really a strategy a human could use.

2 Grids

The idea is that there are two pieces of information that one needs to gain in order to determine the secret code:

1. What is the colour representation?
2. Where does each colour go?

We suggest a systematic approach where one can keep track of all the progress on both questions.

The information at each stage is maintained in a *grid* with 4 rows and 4 columns. Down the side, we keep track of those colours known to present, and their multiplicities: one row for each peg known to be in the secret code. For each colour peg, we use the row to keep track of the positions in which it can be. A cross indicates that the peg is not in that position, a tick that the peg is in that position, and a blank that the information is still pending.

For example, suppose the secret code is 6232. (As usual we will use the numbers 1 up to 6 to stand for the six colours. We also use the notation **B/W** to mean the response of **B** blacks and **W** whites.) Then after the two guesses indicated, the grid is as shown:

1111	0/0	2	×			
1222	2/0	2	×			

and after a third guess, the grid is changed:

3233	2/0	2	×	✓	×	×
		2	×	×		
		3		×		
				×		

It is important to note that, together with the knowledge that the unexplored colours are 4, 5 and 6, **all** the information gained from the first three guesses is displayed in the final grid.

2.1 Exploratories

In order for one to be able to assimilate all the information, the guess must be sufficiently simple. We define an *exploratory* as a guess with one peg of one colour, called the *Trial colour*, and the remaining pegs of another colour, called the *Fill colour*, such that it is known whether the Trial colour is in the secret code or not. If the trial colour is known to be in the secret code, it is called a *player*; otherwise it is called a *dummy*. The position containing the trial colour we call the *Trialpos*. In the above example, the second and third guesses are exploratories.

A response to an exploratory can be fully interpreted. That is, the following questions can be answered:

1. How many of the Fill colour are in the secret code?
2. Which of the Fill or Trial colour is in the Trialpos in the secret code?

The first question is simply a matter of counting the total number of pegs and subtracting 1 if the Trial colour is a player. For instance, in the above example, the third guess receives two pegs, but one of those is for the Trial. Note

that there can be doubt in one particular case: where the Trial colour is a dummy, where no other colour is known to be present, and the response is **3/0**. Then the secret code can possibly be all Fill.

To answer the second question, one looks only at the number of whites according to the following table:

whites	Trial colour is player	Trial colour is dummy
0	Trial in Trialpos	Fill not in Trialpos
1	neither in Trialpos	Fill in Trialpos
2	Fill in trialpos	—

No other number of whites is possible. Again there is a slight exception if the response is **3/0**: then the Fill colour can be in the Trialpos.

For instance, in the above example, the response to the second guess says that the colour 2 is not in the first position.

2.2 Strategy

A strategy for CodeBreaker is thus the following:

1. Start with the guess 1111.
2. Repeatedly use exploratories until the code is known.
3. Guess the known code.

Note that in real life the CodeBreaker can randomly order the colours (and so does not always start with the same colour).

In particular, the next exploratory is determined as follows. The Fill colour is simply the next possibility; if all colours have been used, use any colour known to be absent. The Trial colour is any colour that is known to be present but whose position is not identified, if one exists; otherwise any previous colour acts as a dummy. If the Trial colour is a player then the TrialPos is any possible position for the player; otherwise the TrialPos is any unresolved position.

Thus the previous example might continue:

4424 0/1
3555 0/1

at which stage the secret code is known to be 6232.

2.3 Deduction

Storing the information in a grid allows for considerable but systematic deduction (reminiscent of that used in logic problems in puzzle magazines). For example, if there are three crosses in a row or column, then the fourth position must be a tick. This is used after the fourth guess in the above example to deduce the position of the second 2:

2	×	✓	×	×
2	×	×	×	
3		×		
		×		

This can be generalized; for example, if two rows combined have blanks in only two columns. In the following

grid, it can be deduced that the colour 1 in the first two positions.

1			×	×
1			×	×

Also, colours can be deduced. After the fifth guess in the above example, the presence of one 6 is deduced. Of course, this is the same deduction that happens normally when playing. The advantage here is that deductions are easily spotted in the grid.

2.4 Data

The grid strategy and the extensions discussed in this paper have been coded up. Since the strategy is deterministic, the number of possible games is finite. All data is thus calculated by playing all possible games and averaging the result. (This assumes all secret codes are equally likely; the dropping of this assumption is discussed in Section 4.1.)

2.5 Minor improvements

There are some simple improvements which one can easily implement. A significant improvement can be made by changing the start code. The first two guesses are always 1111 and 1222. So one swops these two guesses around. The improvement is that when the 1222 yields a negative 0/0 response, then one guess is saved. (This represents about a $4^4/6^4 \approx 0.20$ improvement in the overall average number of guesses.)

Another idea is to use in the guess all the pegs whose colour and position are already known, along with the single Trial colour which is known whether to be present or absent, and the remainder the Fill colour. For example, the fourth guess in the above sequence would be 4224. The resultant response is still easy to interpret.

After several responses, often there are exactly two possible codes remaining: the colour representation is known but two positions are unsettled. Then an improvement is to abandon the exploratories and to guess one of the two possible codes. This can be generalised to the situation where all the colours are known and for only two colours are the positions undetermined.

The overall average number of guesses is 5.57. With a bit of practice, this strategy is playable by a human, without pen and paper, and almost instantaneously.

3 Multiple Grids

The main problem with the opening 1111 is that it does not generate much information—it does not splinter the possibilities very well (there are still $5^4 = 625$ possible codes if colour 1 is absent). On the other hand, no other opening guess can be interpreted using a grid. The solution is to use multiple grids.

Consider, for example, starting with the guess 1122.

Say it receives a 1/0 response. Then there are two possibilities: there is a colour 1 in the code, or there is a colour 2 in the code. This can be realised by two grids. Thereafter, exploratories can be interpreted on each grid independently.

Example. Suppose the secret code is 5152. Opening 1122 with 2/0 yields three grids:

1	✓	×	×	×
1	×	✓	×	×
	×	×		
	×	×		

1 1

1			×	×
2	×	×		

2 2

2	×	×	×	✓
2	×	×	✓	×
			×	×
			×	×

2 2

Note that with each grid one must keep track of the colours which could appear more than already determined. For example, in the first grid, the colour 1 can still appear more times. This is indicated by the 1 beneath the possible positions. Such footnotes (or footnumbers?) are added whenever the full quota of a colour is added to the grid; they are added in those positions where extras can appear (the positions in which the colour does not appear in the opening guess).

As the guesses continue, some grids will become inconsistent. Typically (a) a colour is shown not to be in a particular column when it has already assumed to be there, (b) a colour is shown to appear less times than it has already been assumed, or (c) one row or column in the grid is filled with crosses.

Example continued. 1333 receives 0/1 which yields the following; the 1 not appearing in the first position contradicts the first grid in the previous diagram.

1	×	✓	×	×
2	×	×		
		×		
		×		

2	×	×	×	✓
2	×	×	✓	×
3	✓	×	×	×
	×	✓	×	×

2

After that, the next exploratory is 4424 and we are back to one grid.

3.1 Complexity

It is to be noted that the number of grids grows rapidly. A response of 1/1 to 1122 yields four grids; the same response to an opening 1234 yields twelve grids! With five pegs, there can be as many as thirty grids. (In fact, it can be shown that the maximum number of grids grows like $O(3^h)$ where h is the number of positions.)

One can also, in theory, use multiple grids to deal with more complicated guesses at each stage. But this is too

complicated for a human (except possibly if only one grid survives at that moment).

3.2 Strategy

The question is how to choose an exploratory. We use the following direct approach. The Fill colour is as before: simply sequential (starting with the colours that do not appear in the opening guess). For the Trial colour and the TrialPos, look at the sparsest grid and based on that grid determine these values. When at most one (surviving) grid has more than one possibility, then one can switch to the full single-grid strategy on that grid.

The results using the above algorithm are summarised in Table 1. There are only five opening guesses up to symmetry. (Note that the trick with the opening 1111 whereby it is swapped with the next guess is not available with the other openings.)

Opening Guess	1111	1112	1122	1123	1234
Average Guesses	5.57	5.16	5.07	5.15	5.48

Table 1: Multiple grids

3.3 Endgames

After a few guesses, it is reasonably easy for a human to determine from the grids a list of all remaining possibilities. If this list is small enough, one can then play a perfect strategy from there on.

In order to look for the impact, a best splitter approach was implemented. That is, it takes the entire list of current possibilities, and chooses the guess from the list which splits up the set as best as possible according to some simple norm. (See [7, 14] for a comparison of norms.) Specifically the data in Table 2 shows the improvement upon switching to the splitter approach after 3 guesses.

Opening Guess	1111	1112	1122	1123	1234
Average Guesses	5.32	5.00	4.91	4.84	4.92

Table 2: Multiple grids with endgame improvement

Most of the time there is a handful of possibilities left, so this improvement is certainly implementable by a human. (The author as player uses the 1122 opening and the basic exploratory approach to play with an average of around 4.65. The 0.26 improvement over the computer comes from a careful choice of a good exploratory; and the switching over to an almost perfect strategy whenever the number of surviving codes is manageable.)

3.4 Automatic generation of grids

It is not hard to determine, and learn, the initial grid patterns for the dozen or so responses. However, automatic generation of the grids is desirable and was implemented.

One can use the following algorithm: Say the opening guess uses k colours and receives response $\mathbf{b/w}$. Then iterate through all ordered partitions of b into k parts such that each part is at most the corresponding colour occurrence. With each such partition, iterate through all ordered partitions of w such that each part is at most the corresponding colour occurrence minus the part in the partition of b . Then place each combination of partitions on a separate grid as follows. For a colour receiving a black, place crosses in all positions it doesn't appear in; for a colour receiving a white, place crosses in all positions it does appear in. Then use the existing deduction process to weed out impossibilities.

Example. Suppose an opening 1123 receives response 2/1. Then the ordered partitions of the blacks are 200, 110, 101, 011. The partition 200 means that the two correct colours are assumed to be the two 1's. This is recorded by adding two 1's to the empty grid, placing crosses in the third and fourth columns for both these rows, and adding footnotes to those columns. Then the two ordered partitions of the whites are 010 and 001. For the partition 010, the assumption is recorded by adding one 2 and placing a cross in the third column for this row. After deductions, the grid for the combination 200-010 is

1	✓	×	×	×
1	×	✓	×	×
2	×	×	×	✓
	×	×	✓	×

1

On the other hand, the partition 011 of the blacks and partition 100 of the whites produce a contradiction. (This can be seen by observing that if the colours 2 and 3 are in the right position, then the 1 peg cannot be in the wrong position.) The grid, before deduction, is as follows:

1	×	×		
2	×	×		×
3	×	×	×	

23 23 3 2

4 Further Thoughts

4.1 Real adversaries

In employing this strategy against a thinking CodeSetter, the CodeBreaker obviously has the power to randomly permute the colours and the positions, making many secrets isomorphic. Indeed, the only thing the CodeSetter has control over is the partition of 4 given by the colour usage: she has the same five choices as the CodeBreaker. The data in Table 3 gives the result for the five choices of each player, assuming the CodeBreaker uses the strategies analyzed in Table 1.

This data can now be thought of as a pay-off matrix in a zero-sum game. In this form, it can be calculated that the optimal strategy for the CodeBreaker is to use the strategy starting with 1112 3% of the time, the strategy starting 1122 18% of the time, and the strategy starting 1123

		CodeSetter				
		1111	1112	1122	1123	1234
CodeBreaker	1111	5.33	5.09	5.32	5.54	5.86
	1112	5.83	4.94	5.36	5.21	5.07
	1122	5.83	5.23	4.94	5.03	5.13
	1123	5.00	4.92	5.22	5.15	5.19
	1234	4.33	5.01	5.12	5.38	5.50

Table 3: Playing against a real adversary

79% of the time. The average number of guesses needed is 5.175.

4.2 Bigger games

A popular version of Mastermind is one with 5 positions and 8 colours. This has $8^5 = 32768$ possible codes. The data for the grid player (without endgame enhancement) on this game is given in Table 4. The optimal strategy is not known. The data is compared with a random player (randomly pick any remaining possible code).

Opening Guess	Average Number of Guesses
Random	5.86
11111	7.61
11112	7.18
11122	7.00
11123	7.07
11223	7.05
11234	7.45
12345	7.73

Table 4: The grid player on the 5-position 8-colour game

4.3 Even bigger games

Unfortunately, theoretical considerations show that the grid approach is less suitable for other variations of the game (more pegs or more colours), in that the gap between a perfect player and a grid player grows.

For example, consider the 4-position c -colour game for c large. Then some probability shows that the grid approach will take on average $4c/5 - O(1)$ guesses to determine the colour composition of the secret. However, the optimum strategy (see [6]) takes $c/4 + O(1)$ guesses.

Suppose that the number of positions grows too—say we have a h -position h -colour game. Then it can be argued that a secret without repeated colours will take $O(h^2)$ guesses using the grid approach. However, the optimal number of guesses is known to be no worst than $O(h \log h)$ (see [3]).

5 Conclusions

The single-grid approach yields a reasonable human player, especially given that it can be played almost without thinking. The multiple-grid player with a switch to a perfect player after three steps yields a good human player.

6 Acknowledgements

The first computer implementation of the single-grid strategy was produced by Stephen J. Clarke [4]. The author would like to thank Michael Atkins, for many fun games and much cricket-watching, and the weather, for the break in the latter during which the grid idea was invented.

References

- [1] Jr. A.M. Lopez. A prolog mastermind program. *J. Recreational Math.*, pages 81–93, 1991.
- [2] Z. Chen, C. Cunha, and S. Homer. Finding a hidden code by asking questions. *Lecture Notes Comp. Sci. (COCOON '96)*, pages 50–55, 1997.
- [3] V. Chvátal. Mastermind. *Combinatorica*, pages 325–329, 1983.
- [4] S. Clarke. Mastermind: A computer based analysis. B.Sc.Hons thesis, University of Natal, Durban, 1999.
- [5] M.M. Flood. Mastermind strategy. *J. Recreational Math.*, pages 194–202, 1985/86.
- [6] W. Goddard. Mastermind revisited. submitted.
- [7] R.W. Irving. Towards an optimum mastermind strategy. *J. Recreational Math.*, pages 81–87, 1978.
- [8] D.E. Knuth. The computer as mastermind. *J. Recreational Math.*, pages 1–6, 1976.
- [9] K. Koyama and T.W. Lai. An interactive knowledge transfer model and analysis of mastermind game. In *Proceedings of 2nd workshop on algorithmic learning theory (ALT 91)*, pages 196–204, 1991.
- [10] K. Koyama and T.W. Lai. Knowledge acquisition from mastermind queries. In *Proceedings of 2nd pacific rim international conference on artificial intelligence (PRICAI '92)*, pages 646–652, 1992.
- [11] K. Koyama and T.W. Lai. An optimal mastermind strategy. *J. Recreational Math.*, pages 251–256, 1993.
- [12] J.J. Merelo, J. Capio, P. Castillo, V.M. Rivas, and G. Romero. Finding a needle in a haystack using hints and evolutionary computation: the case of genetic mastermind. geneura.ugr.es/~jmerelo.
- [13] M. Mitchell. *Mastermind Mathematics*. Key Curriculum Press, 1999.
- [14] E. Neuirth. Some strategies for mastermind. *Z. Oper. Res. Ser. A-B*, pages B257–B278, 1982.
- [15] T.M. Rao, G. Kazin, and D. O'Brien. Algorithms to play mastermind. *ACM SIGART Newsletter*, pages 33–35, 1986.